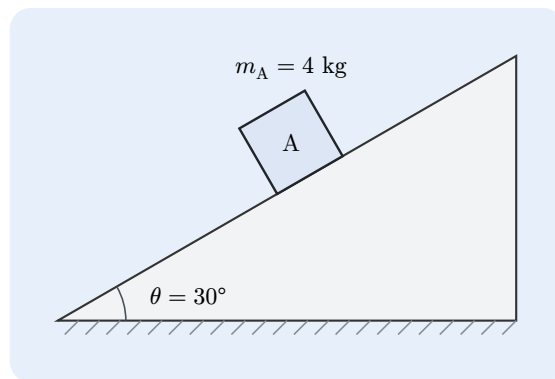


typed-physics

Mechanics and electrical diagrams from physical declarations

User Guide



Version 0.1.0

Contents

1	Getting started	3
1.1	Installation	3
1.2	Your first situation	3
1.3	What you can build	3
2	Quick reference	5
2.1	Situation and surfaces	5
2.2	Bodies	6
2.3	Connectors, loads, and motion	9
2.4	Rigid structures and pendulums	10
2.5	Scene annotations	11
2.6	Figures	12
2.7	Results and force data	12
2.8	Element style builders	13
2.9	Electrical circuits	13
3	Electrical circuits	14
3.1	Declaring a DC circuit	14
3.2	Displaying units	15
3.3	Shaping a parallel network with route:	15
3.4	The return rail	17
3.5	Circuit customization	21
3.6	Electrical scope	22
4	Declaring a situation	23
4.1	Surfaces	23
4.2	Surfaces that meet	24
4.3	Attachment points	25
4.4	Bodies	26
4.5	Friction	28
4.6	Connectors	29
4.7	Applied forces	30
4.8	Velocity	31
4.9	Angular velocity	32
4.10	Drawing-only mechanics elements	33
4.11	Naming quantities	37
5	Figures	38
5.1	<code>scene()</code>	38
5.2	<code>fbf()</code>	45
5.3	<code>components()</code>	46
6	Answers	48
6.1	<code>solve()</code>	48
6.2	<code>force-table()</code>	49
6.3	<code>results()</code>	49
6.4	Numbers and symbols	50
6.5	Choosing the regime with assume:	51
6.6	Bodies by name	52
7	Drawing on top	53
8	Styling	54
8.1	Diagram styles	54
8.2	Element styles	55

9 Errors 57

10 What 0.1.0 does not do 58

11 License 59

1 Getting started

You describe a mechanics problem once, with `situation()`, and ask it for the pieces you want: a figure, a free-body diagram, a worked solution, a single number.

1.1 Installation

Import the package from the Typst preview namespace. A wildcard import gives you every public symbol:

```
1 #import "@preview/typed-physics:0.1.0": *
```

 Typst

Or import only what you need:

```
1 #import "@preview/typed-physics:0.1.0": situation, ramp, block, scene, fbd,
  solve
```

 Typst

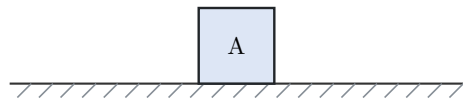
Note. `block` is this package's body constructor, so a wildcard import shadows Typst's own `block`. Reach for the built-in as `std.block`, or import only the names you use.

1.2 Your first situation

Declare a surface, put a body on it, and ask for the figure.

```
1 #let s = situation(
2   ground(length: 6),
3   block("A", mass: 3, on: "ground"),
4 )
5 #scene(s)
```

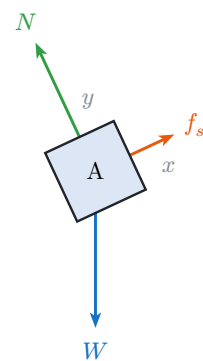
 Typst



Every other view takes the same `s`. This one lists the forces on the block and, since the numbers are all there, draws each arrow in proportion to the force it stands for.

```
1 #let s = situation(
2   ramp("incline", angle: 25deg),
3   block("A", mass: 3, on: "incline",
4     mu: 1.0),
5 )
6 #fbd(s, "A")
```

 Typst



1.3 What you can build

Function	What it gives you
<code>situation(..elements)</code>	The problem. Everything else takes it as its first argument.
<code>ground, wall, ceiling, ramp, arc</code>	Straight and curved surfaces to rest on or against.

block, ball, point-mass, disk, ring	Bodies, including rotating-body outlines.
rod, pivot, support, torque	Drawing vocabulary for rigid-body statics.
pendulum	A suspended bob, string, construction line, and angle.
rope, spring, pulley	Connectors.
force, velocity, angular-velocity	What acts on a body, and how it is already moving or rotating.
dimension	A scene-only distance annotation between anchors or points.
scene(s)	The figure.
fbd(s, name)	A free-body diagram.
components(s, name)	The weight resolved into the surface's axes.
solve(s)	One stated quantity.
force-table(s)	Every force with its components.
results(s)	The solution as a dictionary of numbers.
draw(s)	The scene as CeTZ elements, to draw on top of.
block-style, surface-style, force-style	Build the <code>style:</code> dictionary one element takes. See Section 8 .
theme	The default style dictionary.

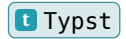
The drawing vocabulary covers straight and curved surfaces, particle and rigid bodies, constraints, connectors, loads, motion, and pendulums. The solver is narrower: it balances one supported block, ball, or point mass on a ground or ramp and states the quantities that fall out. [Section 10](#) lists what it does not do.

2 Quick reference

Every public function and argument is collected here. The sections after this reference explain what the arguments mean and show live examples.

2.1 Situation and surfaces

```
1  #situation(..declarations, gravity: 9.81, style: (:))
2
3  #ground(..name, length: 8, from: none, mu: none, style: (:))
4  #wall(..name, side: left, height: 4, from: none, mu: none, style: (:))
5  #ceiling(..name, length: 8, height: 4, from: none, mu: none, style: (:))
6  #ramp(
7      name,
8      angle: 30deg,
9      length: 6,
10     facing: right,
11     from: none,
12     mu: none,
13     symbol: auto,
14     style: (:),
15 )
16 #arc(
17     name,
18     radius: 3,
19     start-angle: -90deg,
20     end-angle: 90deg,
21     side: "outside",    // or "inside"
22     from: none,
23     mu: none,
24     style: (:),
25 )
```



2.2 Bodies

```
1  #block(  
2    name,  
3    mass: none,  
4    on: none,  
5    at: 50%,  
6    touching: none,  
7    side: right,  
8    hanging: none,  
9    drop: 1.5,  
10   size: 1,  
11   mu: none,  
12   label: auto,  
13   symbol: auto,  
14   style: (:),  
15  )
```

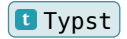
 Typst

```
1  #ball(  
2    name,  
3    mass: none,  
4    on: none,  
5    at: 50%,  
6    touching: none,  
7    side: right,  
8    hanging: none,  
9    drop: 1.5,  
10   radius: 0.5,  
11   mu: none,  
12   label: auto,  
13   symbol: auto,  
14   style: (:),  
15  )
```

 Typst

```
1  #point-mass(  
2    name,  
3    mass: none,  
4    on: none,  
5    at: 50%,  
6    touching: none,  
7    side: right,  
8    hanging: none,  
9    drop: 1.5,  
10   radius: 0.09,  
11   label: auto,  
12   symbol: auto,  
13   style: (:),  
14  )
```

 Typst



```
1  #disk(  
2    name,  
3    mass: none,  
4    on: none,  
5    at: 50%,  
6    touching: none,  
7    side: right,  
8    hanging: none,  
9    drop: 1.5,  
10   radius: 0.5,  
11   radius-mark: true, // or false  
12   orientation: 0deg,  
13   mu: none,  
14   label: auto,  
15   symbol: auto,  
16   style: (:),  
17 )  
18  
19 #ring(  
20   name,  
21   mass: none,  
22   on: none,  
23   at: 50%,  
24   touching: none,  
25   side: right,  
26   hanging: none,  
27   drop: 1.5,  
28   radius: 0.5,  
29   radius-mark: false, // or true  
30   orientation: 0deg,  
31   mu: none,  
32   label: auto,  
33   symbol: auto,  
34   style: (:),  
35 )
```

2.3 Connectors, loads, and motion

```
1 #pulley(name, at: none, radius: 0.4, style: (:))
2 #rope(..name, from: none, to: none, over: none, style: (:))
3 #spring(..name, from: none, to: none, coils: 6, width: 0.28, style: (:))
4
5 #force(on: none, magnitude: none, angle: 0deg, at: auto,
6       label: auto, style: (:))
7 #velocity(on: none, magnitude: none, angle: 0deg, label: auto, style: (:))
8 #angular-velocity(
9     on: none,
10    magnitude: none,
11    direction: "clockwise", // or "counterclockwise"
12    radius: auto,
13    start-angle: auto,
14    end-angle: auto,
15    label: auto,
16    style: (:),
17 )
```

 Typst

2.4 Rigid structures and pendulums

```
1  #rod(  
2      name,  
3      from: none,  
4      to: none,  
5      length: 4,  
6      angle: 0deg,  
7      thickness: 0.16,  
8      mass: none,  
9      center-of-mass: 50%,  
10     label: auto,  
11     symbol: auto,  
12     style: (:),  
13 )  
14 #pivot(name, at: none, radius: 0.12, style: (:))  
15 #support(name, at: none, kind: "pin", angle: 0deg,  
16         size: 0.5, style: (:)) // kind: "roller" or "fixed"  
17 #torque(on: none, at: auto, magnitude: none,  
18        direction: "counterclockwise", radius: 0.65,  
19        label: auto, style: (:)) // or direction: "clockwise"  
20 #pendulum(  
21     name,  
22     from: none,  
23     length: 3,  
24     angle: 20deg,  
25     mass: none,  
26     radius: 0.24,  
27     angle-label: auto,  
28     label: auto,  
29     symbol: auto,  
30     style: (:),  
31 )
```

 Typst

2.5 Scene annotations

```
1  #dimension(  
2    from: none,  
3    to: none,  
4    orientation: "aligned", // "horizontal" or "vertical"  
5    offset: 0.6,  
6    side: auto,             // above/below or left/right  
7    label: auto,  
8    label-position: "center", // "above" or "below"  
9    label-offset: 0.2,  
10   label-rotation: 0deg,  
11   label-fill: white,  
12   arrows: "both",         // "start", "end", or "none"  
13   arrow-tip: "stealth",  
14   arrow-scale: 0.5,  
15   extensions: true,  
16   extension-gap: 0.08,  
17   extension-stroke: auto,  
18   stroke: auto,  
19   color: auto,  
20   text: (:),  
21 )
```

 Typst

2.6 Figures

```

1  #scene(
2      s,
3      labels: "name",      // "symbol", "mass", "both", "none", or overrides
4      angles: "value",     // "symbol", "both", "none", or overrides
5      loads: true,         // true or false
6      frictions: false,    // true or false
7      lengths: false,      // true or false
8      dimensions: (),      // dimension(...) or an array of them
9      forces: none,        // a body name, names, true, or none
10     components: none,     // a body name, names, true, or none
11     style: (:),
12 )
13
14 #draw(
15     s,
16     labels: "name",      // "symbol", "mass", "both", "none", or overrides
17     angles: "value",     // "symbol", "both", "none", or overrides
18     loads: true,         // true or false
19     frictions: false,    // true or false
20     lengths: false,      // true or false
21     dimensions: (),      // dimension(...) or an array of them
22     forces: none,        // a body name, names, true, or none
23     components: none,    // a body name, names, true, or none
24     style: (:),
25 )

```

t Typst

```

1  #fbd(s, name, axes: auto, outline: true, solve: true, style: (:))
2  #components(s, name, of: "weight", style: (:))

```

t Typst

2.7 Results and force data

```

1  #solve(s, ..name, find: auto, direction: true, assume: auto)
2  #force-table(s, ..name, assume: auto)
3  #results(s, ..name, assume: auto)
4  #forces(s, name)

```

t Typst

2.8 Element style builders

```

1  #block-style(fill: auto, stroke: auto, label-text: auto)
2  #surface-style(
3      fill: auto,
4      stroke: auto,
5      hatch-stroke: auto,
6      hatch-spacing: auto,
7      hatch-length: auto,
8  )
9  #force-style(color: auto, stroke: auto, length: auto, text: auto)
10 #connector-style(stroke: auto, fill: auto)

```

 Typst

theme is the exported dictionary containing every diagram-style default.

2.9 Electrical circuits

Electrical names live under the `electricity` namespace so a circuit's voltage-source, resistor, and capacitor vocabulary cannot collide with mechanics declarations.

```

1  #import "@preview/typed-physics:0.1.0": electricity as e
2
3  #e.dc-circuit(source, network, style: (:))
4  #e.voltage-source(name, voltage: none, unit: auto, label: auto, style: (:))
5  #e.resistor(name, resistance: none, unit: auto, label: auto, route: auto, style:
6  (:))
7  #e.capacitor(name, capacitance: none, unit: auto, label: auto, route: auto, style:
8  (:))
9
10 #e.series(..branches, route: auto)
11 #e.parallel(..branches, route: auto)
12
13 #e.diagram(circuit, labels: "both", fold: auto, style: (:))
14 #e.draw(circuit, labels: "both", fold: auto, style: (:))
15
16 #e.resistor-style(symbol: auto, fill: auto, stroke: auto, text: auto)
17 #e.capacitor-style(stroke: auto, text: auto)
18 #e.voltage-source-style(symbol: auto, fill: auto, stroke: auto, text: auto)
19 #e.theme

```

 Typst

3 Electrical circuits

The electricity namespace turns circuit connectivity into a complete drawing. Authors declare which resistors and capacitors are in series and which paths are in parallel; the package chooses positions, branch spacing, wire routes, junctions, source placement, and component rotation.

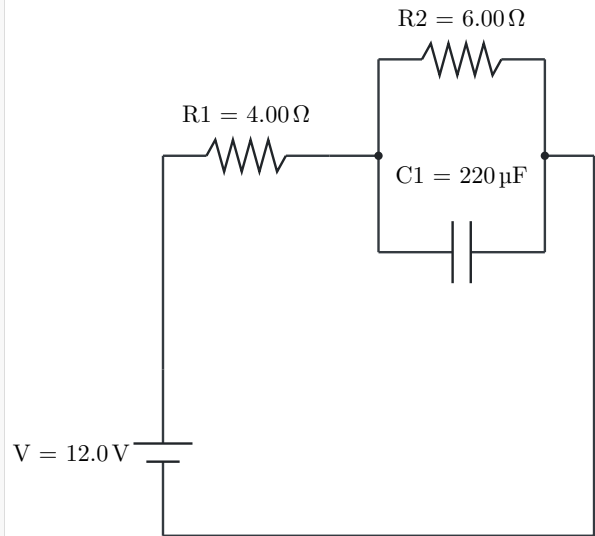
This is intentionally a drawing-only electrical release. Values are carried by the declarations and printed consistently, but they are not evaluated. There is no equivalent-resistance or capacitance, charge, current, voltage-drop, power, transient, Ohm's-law, or Kirchhoff solver yet.

3.1 Declaring a DC circuit

```

1  #let e = electricity
2
3  #let circuit = e.dc-circuit(
4    e.voltage-source("V", voltage: 12),
5    e.series(
6      e.resistor("R1", resistance: 4),
7      e.parallel(
8        e.resistor("R2", resistance: 6),
9        e.capacitor("C1", capacitance:
10         220, unit: "μF"),
11      ),
12    ),
13
14  #e.diagram(circuit)
```

t Typst



`dc-circuit` takes exactly one ideal voltage source and one two-terminal load network. A resistor or capacitor is itself a network, while `series` and `parallel` compose two or more networks and may be nested to any depth.

Function or argument	Meaning
<code>dc-circuit(source, network)</code>	Closes the declared resistor/capacitor network through one ideal DC voltage source.
<code>voltage-source(name, voltage:, unit:)</code>	Declares the source. voltage: is a positive number, symbolic content, or <code>none</code> .
<code>resistor(name, resistance:, unit:)</code>	Declares one resistor. resistance: is a positive number, symbolic content, or <code>none</code> .
<code>capacitor(name, capacitance:, unit:)</code>	Declares one ideal capacitor. capacitance: is a positive number, symbolic content, or <code>none</code> .
<code>series(..branches)</code>	Connects two or more declared networks end to end.
<code>parallel(..branches)</code>	Connects two or more declared networks between the same split and join nodes.
<code>route:</code>	How one branch of a <code>parallel</code> travels between the split and join nodes. See Section 3.3 .

unit:	auto prints volts, ohms, or farads, none omits the unit, and a non-empty string or content supplies a displayed unit such as " μF " or μOmega . It does not rescale the value.
label:	auto uses the component name, none hides it, and a string or content supplies the displayed name.
style:	On dc-circuit , sets defaults for the complete diagram. On a component, overrides only that component.

Every component name must be unique. Zero or negative resistance, capacitance, and voltage are rejected rather than silently changing the component's physical meaning or circuit topology.

3.2 Displaying units

Ohms, farads, and volts are printed when **unit: auto** is left unchanged. A component may instead carry a prefixed or custom display unit. This is annotation rather than conversion: **capacitance: 220, unit: " μF "** prints 220 microfarads and does not reinterpret a value originally expressed in farads.

```

1  #let e = electricity
2
3  #let micro-circuit = e.dc-circuit(
4    e.voltage-source("bias", voltage: 750,
5      unit: " $\mu\text{V}$ "),
6    e.series(
7      e.resistor(
8        "sensor",
9        resistance: 12,
10       unit:  $\mu\text{Omega}$ ,
11       label:  $R_s$ ,
12     ),
13     e.capacitor("coupling", capacitance:
14       470, unit: "nF", label:  $C_c$ ),
15   ),
16   #e.diagram(micro-circuit)
```

The diagram shows a single loop circuit. On the left vertical wire is a voltage source represented by two parallel lines of unequal length, labeled "bias = 750 μV ". On the top horizontal wire is a resistor represented by a zigzag line, labeled " $R_s = 12.0 \mu\Omega$ ". On the right vertical wire is a capacitor represented by two parallel vertical lines, labeled " $C_c = 470 \text{ nF}$ ".

Use a string such as " $\text{k}\Omega$ " or " μF " or mathematical content such as μOmega when the typography should be authored directly. Pass **unit: none** to print only the declared value.

3.3 Shaping a parallel network with route:

Parallel branches are stacked one above another by default, which is the conventional general-purpose presentation. A branch may instead declare how it travels between the split and join nodes, and the declared routes together give the network a frame.

route: value	How the branch travels
auto	The default. The branch gets its own horizontal baseline, joined to the others by the split and join wires.

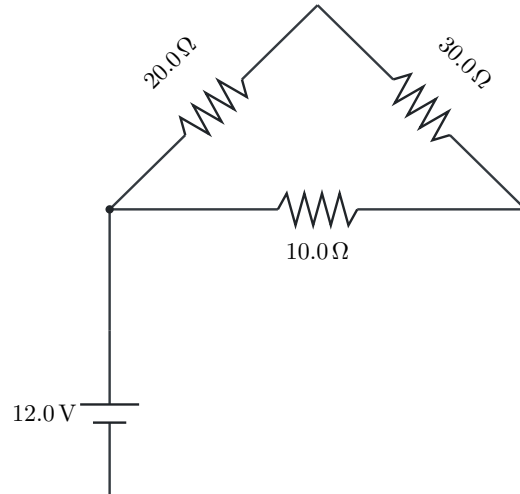
"direct"	Straight from the split node to the join node.
"over"	Around the corner above the direct line.
"under"	Around the corner below it.

A direct branch beside a path routed over the top is the familiar delta:

```

1  #let e = electricity
2
3  #let triangle = e.dc-circuit(
4    e.voltage-source("V", voltage: 12),
5    e.parallel(
6      e.resistor("R1", resistance: 10, route: "direct"),
7      e.series(
8        e.resistor("R2", resistance: 20),
9        e.resistor("R3", resistance: 30),
10       route: "over",
11     ),
12   ),
13 )
14
15 #e.diagram(triangle, labels: "value")

```

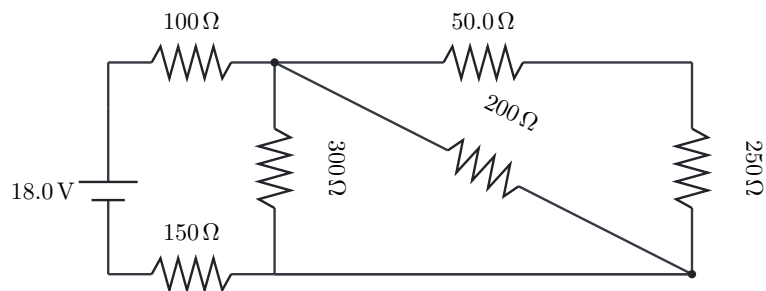
 Typst


The split and join nodes stay level while the frame bulges to one side, so the routed branch peaks midway. A frame carrying branches on **both** sides has no room for that, so its split and join separate vertically and each routed branch turns at a corner instead:

```

1  #let e = electricity
2
3  #let frame = e.dc-circuit(
4    e.voltage-source("V", voltage: 18),
5    e.series(
6      e.resistor("R100", resistance: 100),
7      e.parallel(
8        e.resistor("R300", resistance: 300, route: "under"),
9        e.resistor("R200", resistance: 200, route: "direct"),
10       e.series(
11         e.resistor("R50", resistance: 50),
12         e.resistor("R250", resistance: 250),
13         route: "over",
14       ),
15     ),
16    e.resistor("R150", resistance: 150),
17  ),
18 )
19
20 #e.diagram(frame, labels: "value")

```

 Typst


Every branch still joins the same two electrical nodes; `route`: changes where they are drawn and nothing else. No component receives a position, angle, or endpoint.

A branch routed "over" or "under" turns at one corner, so it holds one component on each leg. Given a single component it takes the first leg and the second is plain wire. A branch routed "direct" spreads its components evenly along the straight run.

Within one `parallel`, either no branch declares a route or all of them do, and at most one branch may take each route. Those rules are what let the declared routes describe a single unambiguous shape.

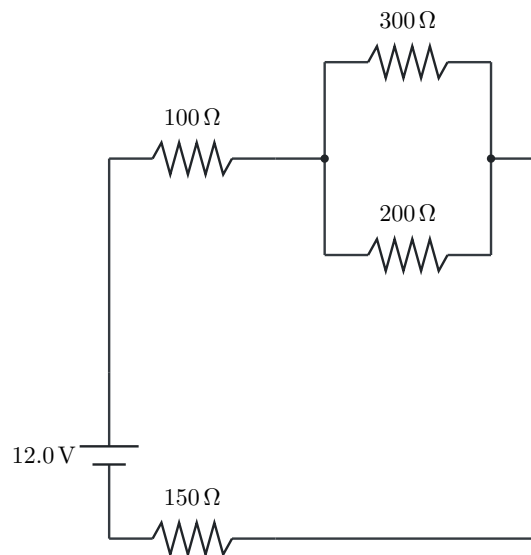
3.4 The return rail

A DC circuit is a loop, and the return rail along its bottom is a place to put components rather than a bare wire. A `series` load is split between the two rails so that they come out as close in width as the branches allow, instead of running the whole load to the right and returning along an empty wire.

```

1  #let e = electricity
2
3  #let circuit = e.dc-circuit(
4    e.voltage-source("V", voltage: 12),
5    e.series(
6      e.resistor("R1", resistance: 100),
7      e.parallel(
8        e.resistor("R2", resistance: 300),
9        e.resistor("R3", resistance: 200),
10     ),
11    e.resistor("R4", resistance: 150),
12  ),
13 )
14
15 #e.diagram(circuit, labels: "value")

```

 Typst


Nothing in the declaration selects a rail. The load is an ordinary series/parallel composition, and R4 reaches the bottom because the loop has already turned by the time the chain gets to it. Pass `fold: false` to keep the whole load on one rail:

```
1  #e.diagram(circuit, labels: "value", fold: false)
```

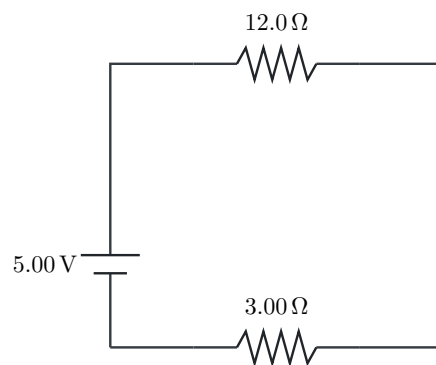
 Typst

A parallel network is the widest thing on the loop, so it anchors the outgoing rail and nothing up to and including it returns. The branches after it are the ones free to move, and a load with no parallel network at all splits wherever the two rails come out closest:

```

1  #let e = electricity
2
3  #e.diagram(
4    e.dc-circuit(
5      e.voltage-source("V", voltage: 5),
6      e.series(
7        e.resistor("R1", resistance: 12),
8        e.resistor("R2", resistance: 3),
9      ),
10   ),
11   labels: "value",
12 )

```

 Typst


Components on the return rail stay in declared order along the loop. Reading the figure clockwise from the source, the bottom rail runs right to left, so the first component to return is the rightmost one.

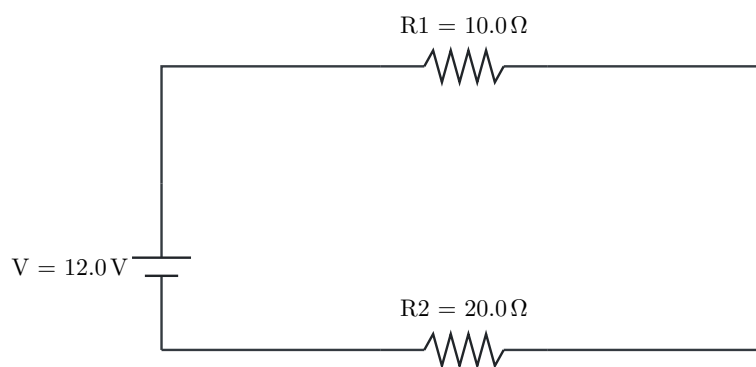
Keeping a small circuit in proportion

The source spans a component length down the left edge, so a load only one component wide on each rail would close as a tall narrow loop. The loop is therefore never drawn narrower than `minimum-loop-width`. Width added that way belongs to neither end of a rail, so it is split evenly and the runs sit centred:

```

1  #let e = electricity
2
3  #let circuit = e.dc-circuit(
4    e.voltage-source("V", voltage: 12),
5    e.series(
6      e.resistor("R1", resistance: 10),
7      e.resistor("R2", resistance: 20),
8    ),
9  )
10
11 #e.diagram(circuit, style: (minimum-loop-width: 8))

```

 Typst


Raise it to spread a small circuit out, or lower it to let one close as tightly as its components allow. A loop already wider than the floor is unaffected, so the setting only reaches the circuits that need it. Like every diagram style key it may be set on `dc-circuit(style:)` for a whole figure or on `diagram(style:)` for one view.

A load that already descends closes along its own exit level rather than dropping below everything. The descending frame in [Section 3.3](#) is placed that way: its lower edge and the circuit's return rail are the same line, which is why `R150` sits on the frame's own bottom side. A frame across a bare source closes the same way, so the source sits on the left edge between the frame's two levels rather than below an empty loop.

That level is used only when the source fits between the load's entry and exit levels and the returning components stay left of wherever the load descends. Otherwise the rail drops below the whole load, which is always available.

`fold: true` asks for the return rail explicitly and reports when the load has no trailing components after a parallel network to put there.

3.5 Circuit customization

```

1  #let e = electricity
2
3  #let circuit = e.dc-circuit(
4    e.voltage-source(
5      "battery",
6      voltage: 6,
7      unit: "mV",
8      style: e.voltage-source-style(
9        symbol: "circle",
10       fill: rgb("#FF3BF"),
11     ),
12   ),
13   e.series(
14     e.resistor(
15       "load",
16       resistance: 7,
17       label: $R_L$,
18       unit: $mu\ \Omega$,
19       style: e.resistor-style(
20         symbol: "rectangle",
21         fill: rgb("#D0EBFF"),
22         stroke: 1.2pt + blue,
23       ),
24     ),
25     e.capacitor(
26       "C1",
27       capacitance: 47,
28       unit: "μF",
29       style: e.capacitor-style(stroke:
30         1.2pt + purple),
31     ),
32     style: (wire-stroke: 1.1pt +
33       rgb("#495057")),
34   )
35   #e.diagram(circuit)

```

t Typst

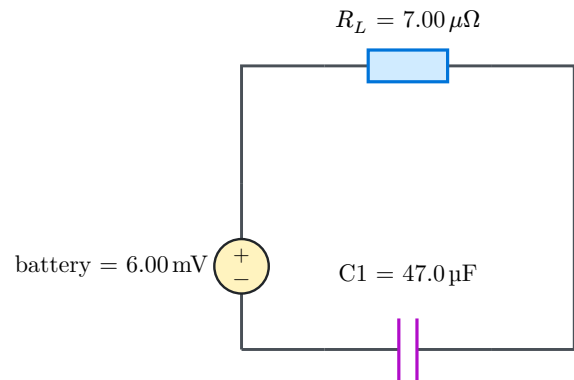


Diagram style key	Meaning
wire-stroke	Wires and component leads.
component-stroke	Default resistor and capacitor symbol stroke.
component-fill	Default fill for the closed rectangular resistor symbol.
source-fill	Default voltage-source fill.

<code>junction-fill, show-junctions</code>	Junction-dot paint and visibility.
<code>resistor-symbol</code>	Default resistor shape: "zigzag" or "rectangle".
<code>voltage-source-symbol</code>	Default source shape: "battery" for long and short plates, or "circle" for a circular source with polarity marks.
<code>component-length</code>	Terminal-to-terminal span reserved for one ordinary component.
<code>resistor-length, resistor-height</code>	Length and height of either resistor symbol.
<code>capacitor-plate-gap, capacitor-plate-height</code>	Separation and height of the two capacitor plates.
<code>source-radius</code>	Radius of the circular ideal voltage source.
<code>source-plate-gap</code>	Separation between the battery symbol's positive and negative plates.
<code>source-long-plate, source-short-plate</code>	Lengths of the battery symbol's positive and negative plates.
<code>parallel-gap, branch-lead</code>	Clearance between parallel paths and the wire length around their split and join.
<code>label-offset, label-text</code>	Distance and text styling for names and values.
<code>source-clearance</code>	Clearance between the load network and the source return path.
<code>minimum-loop-width</code>	Narrowest the complete circuit loop may be drawn. See Section 3.4.1 .
<code>apex-rise</code>	Height of the peak on a level routed frame.
<code>frame-rise</code>	Vertical separation between the split and join nodes of a descending routed frame.
<code>scale</code>	Scale of the complete rendered circuit.

Resistor and voltage-source style builders accept `symbol:`, `fill:`, `stroke:`, and `text:`. `resistor-style(symbol: "rectangle")` and `voltage-source-style(symbol: "circle")` override the default symbol for one declaration. `capacitor-style` accepts `stroke:` and `text:` for the conventional two-plate symbol. Component styles override the diagram defaults for that declaration only.

3.6 Electrical scope

This release covers all two-terminal networks made from ideal resistors and capacitors that `series` and `parallel` can compose. `route:` and the return rail place an already declared topology; they do not change what is connected to what.

Inductors, switches, meters, multiple or dependent sources, nonlinear devices, current arrows, voltage-polarity annotations, and every form of circuit solving are outside this drawing release. Capacitors are drawing declarations only; there is no charge, transient, impedance, or frequency-domain solver.

4 Declaring a situation

```
1 #situation(..elements, gravity: 9.81, style: (:))
```

 Typst

Argument	Meaning
<code>..elements</code>	Surfaces, bodies, and forces, in any order. A body that uses <code>touching:</code> must come after the body it touches.
<code>gravity:</code>	The acceleration due to gravity. A number in m/s^2 prints as g and substitutes to itself; content such as <code>\$g\$</code> keeps it symbolic.
<code>style:</code>	Style overrides applied to every view of this situation. See Section 8 .

You never write a coordinate. A body names the surface it rests on and how far along it sits, so changing a ramp's angle moves the block, its rotation, the hatching, and the angle arc together.

Lengths are CeTZ canvas units, which are centimetres on the page. A ramp with `length: 6` is six centimetres along its incline.

4.1 Surfaces

```
1 #ground(name, length: 8, mu: none, style: (:))
2 #wall(name, side: left, height: 4, mu: none, style: (:))
3 #ceiling(name, length: 8, height: 4, mu: none, style: (:))
4 #ramp(name, angle: 30deg, length: 6, mu: none, symbol: auto, style: (:))
```

 Typst

The name is optional for `ground`, `wall`, and `ceiling` they default to "ground", "wall", and "ceiling". Name a ramp yourself, since a situation may hold more than one.

Argument	Meaning
<code>length:</code>	How long the surface is. For a ramp, measured along the incline rather than along its base.
<code>height:</code>	How high a wall stands, or how far above the ground a ceiling sits.
<code>side:</code>	Which end of the scene a wall stands at, <code>left</code> or <code>right</code> .
<code>angle:</code>	A ramp's inclination, between <code>0deg</code> and <code>90deg</code> .
<code>facing:</code>	Which way a ramp climbs, <code>right</code> or <code>left</code> .
<code>from:</code>	Where this surface begins: an attachment point on a surface declared before it. See Section 4.2 .
<code>mu:</code>	Friction for everything that touches this surface. See Section 4.5 .
<code>symbol:</code>	What the inclination is called in the algebra. Defaults to θ . See Section 4.11 .
<code>style:</code>	How this surface is drawn. See Section 8 .

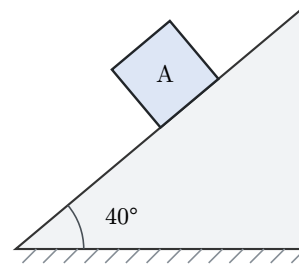
A ramp rises from its foot in the direction `facing:` names, with the ground drawn under it.

```

1 #let s = situation(
2   ramp("incline", angle: 40deg, length:
3     5),
4   block("A", mass: 2, on: "incline", at:
5     60%),
6 )
7 #scene(s)

```

t Typst

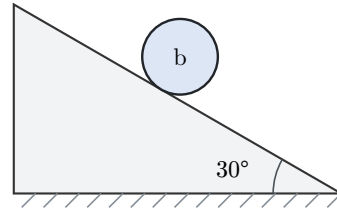


```

1 #let s = situation(
2   ramp("slope", angle: 30deg, length: 5,
3     facing: left),
4   ball("b", mass: 1, on: "slope", at:
5     55%),
6 )
7 #scene(s)

```

t Typst



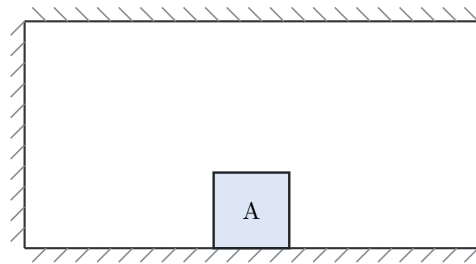
Walls and a ceiling bound the scene. Bodies can rest against them, but 0.1.0 cannot solve a body they support.

```

1 #let s = situation(
2   ground(length: 6),
3   wall(side: left, height: 3),
4   ceiling(length: 6, height: 3),
5   block("A", mass: 2, on: "ground"),
6 )
7 #scene(s)

```

t Typst



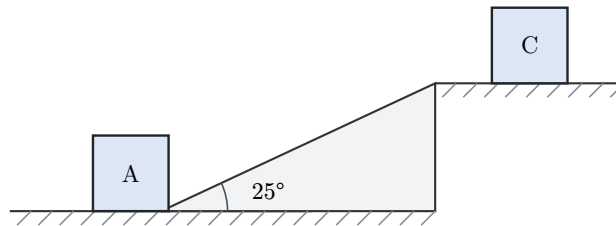
4.2 Surfaces that meet

from: places a surface at an attachment point of a surface declared before it, which is how surfaces compose: a ramp whose foot sits at the end of a ground, a ground that begins at a ramp's apex, a valley between two slopes. Without it every surface starts at the origin.

```

1  #let s = situation(
2    ground("approach", length: 4),
3    ramp("incline", angle: 25deg, length: 4,
4      from: "approach"),
5    ground("shelf", length: 2.5,
6      from: "incline.apex"),
7    block("A", mass: 2, on: "approach", at: 40%),
8    block("C", mass: 1, on: "shelf", at: 50%),
9  )
10 #scene(s)

```

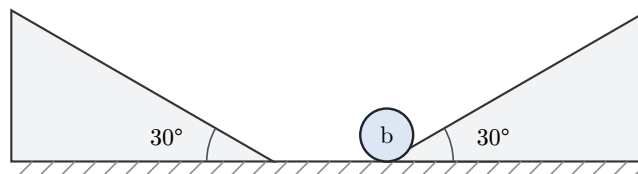
 Typst


Two ramps and a floor make a valley.

```

1  #let s = situation(
2    ramp("left-slope", angle: 30deg, length: 4,
3      facing: left),
4    ground("floor", length: 3,
5      from: "left-slope.foot"),
6    ramp("right-slope", angle: 30deg, length: 4,
7      from: "floor"),
8    ball("b", mass: 1, on: "floor", radius: 0.35),
9  )
10 #scene(s)

```

 Typst


A wall with no from: still stands at the edge of everything placed before it.

4.3 Attachment points

Anywhere one element pins to another, the same three spellings work:

Written as	Means
"ceiling"	That element's default anchor: the midpoint of a surface, the centre of a body or pulley.

"incline.apex"	A named anchor of that element.
(on: "ceiling", at: 40%)	A point that far along a surface.

Element	Anchors
surfaces	start, end, surface
ramps also	foot, apex, base
bodies	center, contact, top, bottom, left, right, uphill, downhill, outward
pulleys	center, top, bottom, left, right

An element may only attach to something declared before it.

4.4 Bodies

```

1 #block(name, mass: none, on: none, at: 50%, touching: none, side: right,
2     hanging: none, drop: 1.5, size: 1, mu: none, label: auto,
3     symbol: auto, style: (:))
4 #ball(name, ..., radius: 0.5)
5 #point-mass(name, ..., radius: 0.09)
```

 Typst

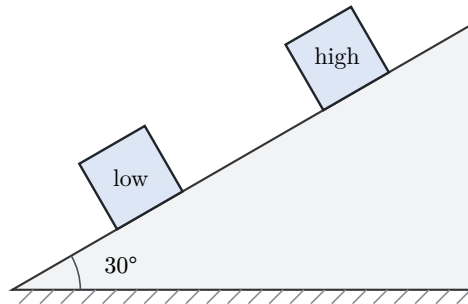
All three take the same placement arguments and differ only in what is drawn.

Argument	Meaning
mass:	A number in kilograms, or content such as m to keep it symbolic.
on:	The surface it rests on.
at:	How far along that surface, as a ratio. Defaults to 50%.
touching:	Rest it against another body instead of positioning it by ratio.
side:	Which side of that body, left or right.
size:	The edge of a square, or a (width, height) pair measured along the surface and out of it.
radius:	The radius of a ball or the dot of a point mass.
radius-mark:	Whether the centre dot and radial spoke are shown. Defaults to true for a disk and false for a ring.
orientation:	Where an enabled radial spoke points, relative to the supporting surface.
hanging:	Hang it below an attachment point instead of resting it on anything.
drop:	How far below that point it hangs.
mu:	Friction for the contact this body makes. See Section 4.5 .
label:	What to draw inside the body, instead of its name.
symbol:	What the mass is called in the algebra. See Section 4.11 .
style:	How this body is drawn. See Section 8 .

Give a body on:, touching:, or hanging:.

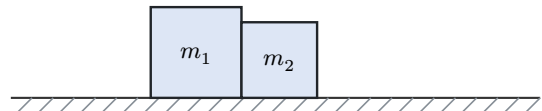
at: measures along the surface, from its start.

```
1 #let s = situation(
2   ramp("incline", angle: 30deg, length:
3     7),
4   block("low", mass: 1, on: "incline",
5     at: 30%),
6   block("high", mass: 1, on: "incline",
7     at: 75%),
8 )
9 #scene(s)
```



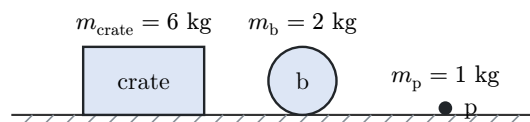
touching: rests a body against another one. Their sizes need not match.

```
1 #let s = situation(
2   ground(length: 7),
3   block("m1", mass: 2, on: "ground", at:
4     35%,
5     size: 1.2, label: $m_1$),
6   block("m2", mass: 3, on: "ground",
7     touching: "m1", side: right, label:
8     $m_2$),
9 )
10 #scene(s)
```



A rectangle, a ball, and a point mass.

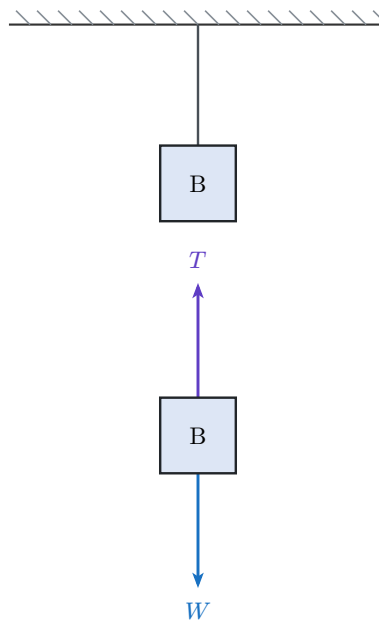
```
1 #let s = situation(
2   ground(length: 7),
3   block("crate", mass: 6, on: "ground",
4     at: 25%, size: (1.6, 0.9)),
5   ball("b", mass: 2, on: "ground",
6     at: 55%, radius: 0.45),
7   point-mass("p", mass: 1, on: "ground",
8     at: 82%),
9 )
10 #scene(s, labels: "both")
```



hanging: holds a body below an attachment point. It rests on nothing, so it stays upright and the solver will not take it.

```
1 #let s = situation(
2   ceiling(length: 5, height: 3),
3   block("B", mass: 2, drop: 1.6,
4     hanging: (on: "ceiling", at: 50%)),
5   rope(from: (on: "ceiling", at: 50%),
6     to: "B.top"),
7 )
8 #scene(s)
9 #fbd(s, "B")
```

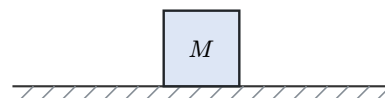
t Typst



label: replaces the text drawn inside the body, in the scene and in the free-body diagram.

```
1 #let s = situation(
2   ground(length: 5),
3   block("crate", mass: 12, on: "ground",
4     label: "$M$"),
5 )
6 #scene(s)
```

t Typst



A block that would extend past the end of its surface stops compilation:

```
1 #block("A", mass: 4, on: "incline", at: 95%)
```

t Typst

```
1 error: assertion failed: typed-physics: block "A" hangs off the end of
2   "incline" by 0.150 – move it back with `at:` or shrink it with `size:`
```

d Text

4.5 Friction

Surfaces and bodies both take μ :, in four forms:

Form	Meaning
<code>mu: 0.3</code>	One coefficient for both static and kinetic friction.
<code>mu: (s: 0.4, k: 0.3)</code>	Static and kinetic given separately.
<code>mu: (static: 0.4, kinetic: 0.3)</code>	The same, spelled out.
<code>mu: μ</code>	A symbol, which keeps the coefficient in the algebra.

Give only `s:` or only `k:` and that value is used for both. A contact with no `mu:` is frictionless.

Note. A body's `mu:` describes the contact it makes; a surface's `mu:` describes everything that touches it. The nearer declaration wins, so a rough ramp can still carry one frictionless body.

4.6 Connectors

```
1 #rope(name, from: none, to: none, over: none, style: (:))
2 #spring(name, from: none, to: none, coils: 6, width: 0.28, style: (:))
3 #pulley(name, at: none, radius: 0.4, style: (:))
```

 Typst

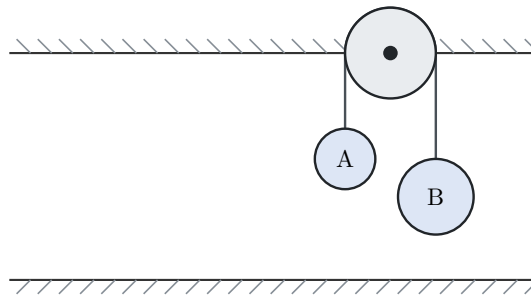
Argument	Meaning
<code>from:, to:</code>	The attachment points a connector spans. See Section 4.3 .
<code>over:</code>	A pulley the rope runs over. It leaves each end at a tangent and wraps the wheel.
<code>coils:, width:</code>	How many zigzags a spring has, and how far they swing.
<code>at:</code>	Where a pulley sits.
<code>radius:</code>	How big the wheel is.

Connectors are drawn, not solved. A rope tells the free-body diagram that a tension acts along it, with a magnitude the package leaves open.

```

1  #let s = situation(
2    ground(length: 7, mu: 0.2),
3    ceiling(length: 7, height: 3),
4    pulley("P", at: (on: "ceiling", at: 72%), radius : 0.6),
5    ball("A", mass: 4, hanging: "P.left", drop : 1, radius : 0.4),
6    ball("B", mass: 2, hanging: "P.right",
7      drop: 1.4),
8    rope(from: "A.top", to: "B.top", over: "P"),
9  )
10 #scene(s)

```

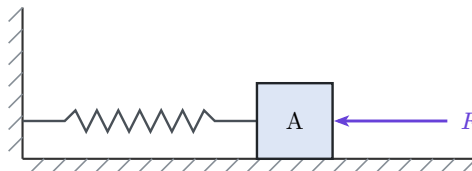
 Typst


)

```

1  #let s = situation(
2    ground(length: 6),
3    wall(side: left, height: 2),
4    block("A", mass: 3, on: "ground", at: 60%),
5    spring(to: "A.left", coils: 7,
6      from: (on: "wall", at: 25%)),
7    force(on : "A", magnitude : 10, angle : 180deg)
8  )
9  #scene(s)

```

 Typst


4.7 Applied forces

```

1  #force(on: none, magnitude: none, angle: 0deg, at: auto,
2    label: auto, style: (:))

```

 Typst

Argument	Meaning
on:	The body or rod the force acts on.

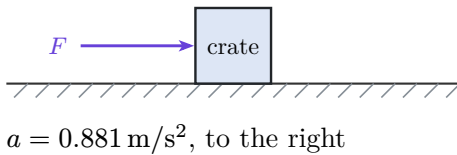
magnitude:	A number in newtons, or content such as F .
angle:	Measured from the horizontal, like every angle in the package.
at:	For a rod, the ratio along it where the force is applied. Defaults to its centre.
label:	The symbol drawn beside the arrow. Defaults to F , numbered when a body carries several.
style:	How this arrow is drawn. See Section 8 .

The arrow arrives at the face the force acts on and points the way the force points.

```

1 #let s = situation(
2   ground(length: 6, mu: 0.25),
3   block("crate", mass: 12, on: "ground"),
4   force(on: "crate", magnitude: 40, angle: 0deg),
5 )
6 #scene(s)
7 #solve(s)
```

 Typst

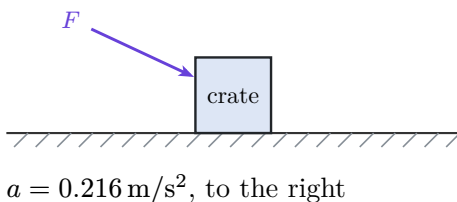


An angled push presses the body into the surface, raising the normal force and with it the friction.

```

1 #let s = situation(
2   ground(length: 6, mu: 0.25),
3   block("crate", mass: 12, on: "ground"),
4   force(on: "crate", magnitude: 40, angle: -25deg),
5 )
6 #scene(s)
7 #solve(s)
```

 Typst



4.8 Velocity

```
1 #velocity(on: none, magnitude: none, angle: 0deg, label: auto, style: (:))
```

 Typst

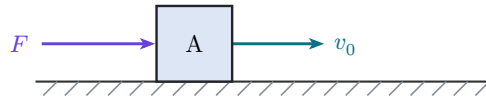
How fast a body is already moving. A velocity is stated by you rather than derived, and is drawn in its own colour so it is never mistaken for a force. It never appears in a free-body diagram.

```

1 #let s = situation(
2   ground(length: 6, mu: 0.25),
3   block("A", mass: 3, on: "ground", at:
4     35%),
5   velocity(on: "A", magnitude: 5,
6     label: $v_0$),
7   force(on: "A", magnitude: 10, angle:
8     0deg),
9 )
10 #scene(s)

```

t Typst



4.9 Angular velocity

```

1 #angular-velocity(
2   on: none,
3   magnitude: none,
4   direction: "clockwise",
5   radius: auto,
6   start-angle: auto,
7   end-angle: auto,
8   label: auto,
9   style: (:),
10 )

```

t Typst

An angular-velocity arrow hugs the outside of a ball, disk, or ring. **direction:** determines which way its arrowhead travels. **radius: auto** clears the body's outline, while an explicit radius controls the gap. **start-angle:** and **end-angle:** are raw world angles that choose which free part of the circle carries the arrow; their winding is adjusted to agree with **direction:**.

Argument	Meaning
on:	The round body whose rotation is being annotated.
magnitude:	An optional angular-speed declaration carried with the annotation; label: controls what is printed.
direction:	"clockwise" or "counterclockwise".
radius:	The arrow radius from the body's centre, or auto to clear the outline.
start-angle: , end-angle:	The raw polar angles delimiting the visible curved arrow.
label:	The arrow label, defaulting to ω .
style:	Motion colour, stroke, and text overrides through force-style() .

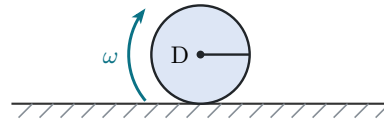
For example, **style: force-style(color: orange.darken(20%))** changes only this curved arrow while preserving the scene's other motion styling.

The familiar rolling-wheel arrow occupies the free space to the left and points upward for clockwise motion.

```

1  #let s = situation(
2    ground(length: 5),
3    disk("D", on: "ground", radius: 0.65),
4    angular-velocity(
5      on: "D",
6      direction: "clockwise",
7      radius: 0.95,
8      start-angle: 220deg,
9      end-angle: 140deg,
10     label:  $\omega$ ,
11   ),
12 )
13 #scene(s)
```

 Typst



4.10 Drawing-only mechanics elements

The elements in this section participate in relational placement and `scene()`, but deliberately do not add equations to `solve()`. They let a problem statement show the full geometry while the current solver keeps its narrower, honest scope.

Loop-the-loop

`arc()` is a curved support. Its start point is the origin or `from:`, and its polar-angle sweep determines the rest of the track. `side:` chooses whether bodies occupy the outside of the curve or its inside.

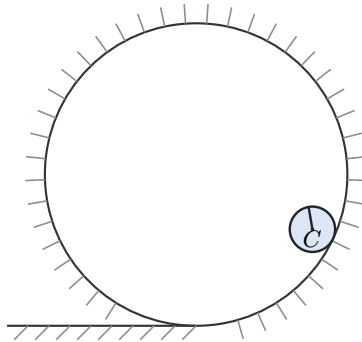
The familiar loop-the-loop uses an inside support. The disk stays tangent to the track as `at:` moves it around the circle.

```

1  #let s = situation(
2    ground("approach", length: 2.5),
3    arc("loop", radius: 2,
4      start-angle: -90deg,
5      end-angle: 270deg,
6      side: "inside",
7      from: "approach.end"),
8    disk("car", mass: 1, on: "loop", at: 18%,
9      radius: 0.3, orientation: 35deg,
10     label: $C$),
11  )
12  #scene(s)

```

t Typst



The arc's `start`, `end`, `surface`, and `center` anchors are available by name. A ratio attachment such as (`on: "loop", at: 75%`) reaches any point along the curve. Where an arc meets a straight foundation, straight hatches take priority and nearby curved hatches wait until their full length fits clear of the junction.

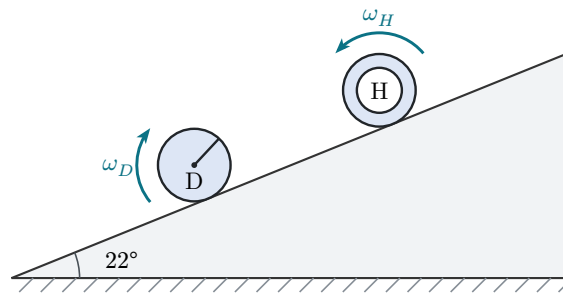
Disk versus hoop

`disk()` and `ring()` take the ordinary body-placement arguments plus `radius-mark:`. It defaults to `true` for a solid disk and `false` for a ring, whose open centre reads more cleanly without it. When enabled, the centre dot and radial spoke run all the way to the body's rim. `orientation:` controls where the spoke points. The name inside the body automatically chooses the nearest horizontal or vertical position opposite the spoke, leaving the mark clear. This lets the classic rolling-race setup show orientation without claiming to calculate angular acceleration.

```

1  #let s = situation(
2    ramp("race", angle: 22deg, length: 8),
3    disk("D", mass: 2, on: "race", at: 35%,
4      radius: 0.48, orientation: 25deg),
5    ring("H", mass: 2, on: "race", at: 68%,
6      radius: 0.48),
7    angular-velocity(on: "D",
8      direction: "clockwise", label:  $\omega_D$ ),
9    angular-velocity(on: "H",
10     direction: "counterclockwise",
11     start-angle: 40deg, end-angle: 140deg,
12     label:  $\omega_H$ ),
13  )
14  #scene(s)

```

 Typst


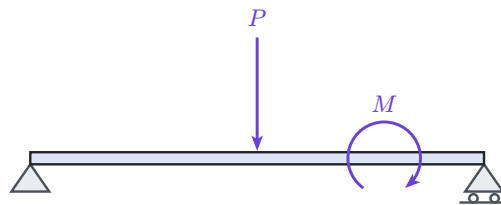
Simply supported beam

`rod()` begins at the origin or an attachment. Give `to:` to span two existing anchors, or give `length:` and `angle:`. Its anchors are `start`, `end`, `center`, and `center-of-mass`.

`support()` draws a "pin", "roller", or "fixed" constraint; `pivot()` draws a hinge; and `torque()` draws an applied clockwise or counterclockwise couple. A force on a rod accepts `at:` as a ratio along its length.

The central point-load beam is the standard first statics problem: one pin, one roller, and loads whose reactions would be found from force and moment balance.

```
1  #let s = situation(
2    rod("beam", length: 6, mass: 12, label: none),
3    support("A", at: "beam.start", kind: "pin"),
4    support("B", at: "beam.end", kind: "roller"),
5    force(on: "beam", at: 50%, magnitude: 20,
6      angle: -90deg, label: $P$),
7    torque(on: "beam", at: 78%,
8      direction: "clockwise", radius: 0.48,
9      label: $M$),
10 )
11 #scene(s)
```

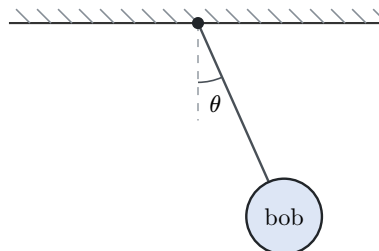


Use (on: "beam", at: 35%) wherever an attachment needs a point between the named rod anchors. angle: on a support rotates its foundation, which is useful for wall-mounted and inclined constraints.

Galileo's pendulum

pendulum() combines a pivot, string, bob, vertical construction line, and angular marker. Its angle is measured from downward vertical, positive towards the right. Set angle-label: none to omit the angular label.

```
1  #let s = situation(
2    ceiling(length: 5, height: 4),
3    pendulum("bob",
4      from: (on: "ceiling", at: 50%),
5      length: 2.8, angle: 24deg,
6      mass: 1.5, radius: 0.5),
7  )
8  #scene(s)
```



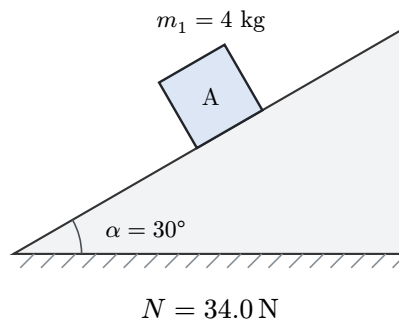
The pendulum exposes `pivot`, `bob`, and `center` anchors, so a connector or later drawing element can attach to either end.

4.11 Naming quantities

Masses print as m subscripted with the body's name, and a lone ramp's inclination prints as θ . Pass `symbol:` to name either yourself. The name you give is used in the figure and in every equation.

```
1 #let s = situation(
2   ramp("incline", angle: 30deg, length: 6,
3     symbol: $alpha$),
4   block("A", mass: 4, on: "incline",
5     at: 55%, symbol: $m_1$),
6 )
7 #scene(s, labels: "both", angles: "both")
8 #solve(s, find: "normal")
```

 Typst



A situation with two ramps tells them apart by name, so their angles print as θ_{lower} and θ_{upper} unless you say otherwise.

5 Figures

5.1 scene()

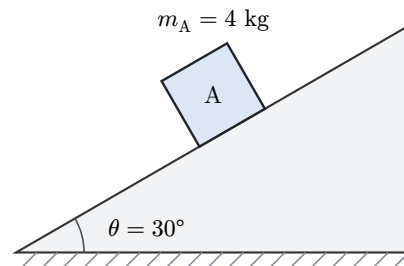
```
1 #scene(s, labels: "name", angles: "value", loads: true, frictions: false,
2     lengths: false, dimensions: (), forces: none, components: none,
3     style: (:))
```

t Typst

Argument	Meaning
labels:	What to write on a body: "name", "symbol", "mass", "both", or "none" or a mode with per-body overrides.
angles:	How to mark a ramp's inclination: "value" (30°), "symbol" (ϑ), "both" ($\vartheta = 30^\circ$), or "none" or a mode with per-ramp overrides.
loads:	Whether to draw the applied forces.
frictions:	Whether to write each contact's coefficients beside the body that makes it.
lengths:	Whether to write each surface's length beside it, as $\ell = 7$.
dimensions:	One <code>dimension(...)</code> annotation or an array of them.
forces:	Draw the forces on a body where it sits: a name, several names, or <code>true</code> for every body.
components:	Resolve the weight on a body where it sits: a name, several names, or <code>true</code> for every body.
style:	Style overrides for this view. See Section 8 .

```
1 #let s = situation(
2     ramp("incline", angle: 30deg, length:
3         6),
4     block("A", mass: 4, on: "incline", at:
5         55%),
6 )
7 #scene(s, labels: "both", angles: "both")
```

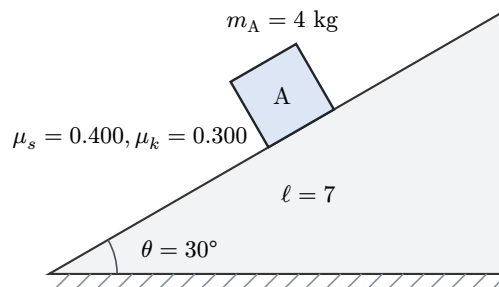
t Typst



Turn everything on when the figure has to carry the problem statement by itself.

```
1 #let s = situation(
2   ramp("incline", angle: 30deg, length: 7),
3   block("A", mass: 4, on: "incline", at: 55%,
4     mu: (s: 0.40, k: 0.30)),
5 )
6 #scene(s, labels: "both", angles: "both",
7   frictions: true, lengths: true)
```

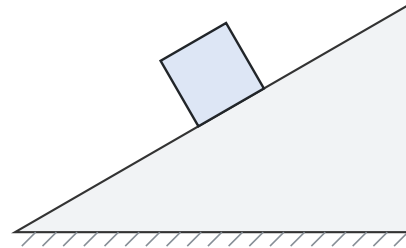
t Typst



Turn everything off for a figure the surrounding text already explains.

```
1 #let s = situation(
2   ramp("incline", angle: 30deg, length:
3     6),
4   block("A", mass: 4, on: "incline", at:
5     55%),
6 )
7 #scene(s, labels: "none", angles: "none")
```

t Typst



Configurable dimensions

`dimension()` draws a drafting-style measurement in one `scene()` or `draw()` call; it is not part of the physical `situation()`. Its `from:` and `to:` accept the package's ordinary anchors, connector anchors such as `"spring.start"`, or raw `(x, y)` points.

orientation: `"aligned"` measures along the segment joining the anchors. `"horizontal"` measures their x-separation, and `"vertical"` measures their y-separation. The axis-aligned modes project both anchors orthogonally, so an incline's run and rise do not require invented coordinates.

offset: moves the line away from the measured points. An aligned dimension accepts any `side:`. A horizontal dimension uses `"above"` or `"below"`, while a vertical dimension uses `"left"` or `"right"`. When omitted, `side:` auto chooses `"above"` for aligned and horizontal dimensions and `"right"` for vertical ones. Dashed projection lines retain the connection to the measured points. Each projection stops before the first body, rod, or straight surface it meets, so it does not draw through a target outline.

label-position: `"center"` follows the drafting convention: the label interrupts the dimension line, and its padded gap grows with the label's content and text size. `"above"` and `"below"` place the label beside an uninterrupted line.

Argument	Meaning
----------	---------

from:, to:	Named anchors, connector anchors, ratio attachments, or raw coordinate pairs.
orientation:	"aligned", "horizontal", or "vertical".
offset:	Distance from the measured points to the dimension line.
side:	auto, "above", "below", "left", or "right".
label:	Math, text, none, or auto for L .
label-position:	"center", "above", or "below".
label-offset:	Distance from the line for an above or below label.
label-rotation:	Rotation applied only to the label.
label-fill:	Background behind a centered label; defaults to white.
arrows:	Arrowheads on "both", "start", "end", or "none".
arrow-tip:	A CeTZ mark name, such as "stealth" or "triangle".
arrow-scale:	Scale factor for both arrowheads.
extensions:	Whether the measured points are projected to the offset line.
extension-gap:	Clearance left before the first target or obstruction.
extension-stroke:	Stroke for the projection lines; dashed by default.
stroke:, color:	Line weight/style and colour; colour defaults to grey.
text:	Text arguments for this label, such as (size: 11pt).

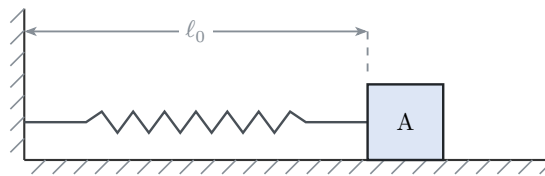
A spring's own anchors make its natural length straightforward to annotate.

```

1  #let s = situation(
2    ground(length: 7),
3    wall(side: left, height: 2),
4    block("A", on: "ground", at: 72%),
5    spring("coil",
6      from: (on: "wall", at: 25%),
7      to: "A.left", coils: 7),
8  )
9  #scene(
10   s,
11   dimensions: dimension(
12     from: "coil.start",
13     to: "coil.end",
14     offset: 1.2,
15     label:  $\ell_0$ ,
16     label-position: "center",
17     extensions : true
18   ),
19 )

```

t Typst



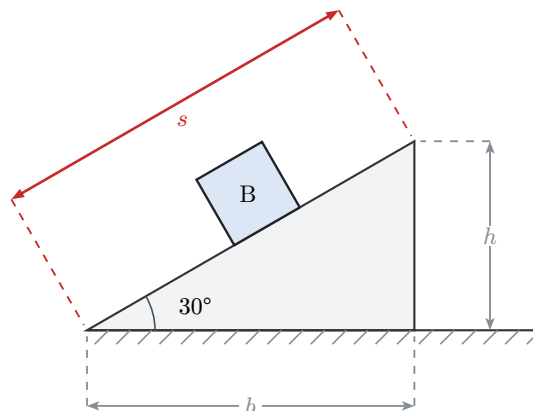
The same anchors can describe the inclined length and its vertical rise.

```

1  #let s = situation(
2    ramp("incline", angle: 30deg, length: 5),
3    block("B", on: "incline", at: 55%),
4    ground(length : 6)
5  )
6  #scene(
7    s,
8    dimensions: (dimension(
9      from: "incline.foot",
10     to: "incline.apex",
11     offset: 2,
12     label: $$s$,
13     label-position: "below",
14     arrow-tip: "triangle",
15     stroke: 1pt,
16     color: rgb("#C92A2A"),
17   ),
18   dimension(
19     from: "incline.apex",
20     offset : 1,
21     to: "ground",
22     orientation: "vertical",
23     side: "right",
24     label: $$h$,
25   ),
26   dimension(
27     from: "incline.foot",
28     offset : 1,
29     to: "incline.base",
30     orientation: "horizontal",
31     side: "below",
32     label: $$b$,
33   )
34 ),
35 )

```

t Typst



Placing individual annotations

The string forms remain the concise default. A dictionary keeps that global `mode:` and adds `overrides:` for individual bodies or ramps.

Each override names its element with `on:`. `offset:` is a raw (x , y) displacement in diagram world units, added to the automatically chosen position, so $(0, 0)$ changes nothing. `rotation:` rotates only the label. `visible:` `false` hides that one annotation, and an override may carry its own `mode:`.

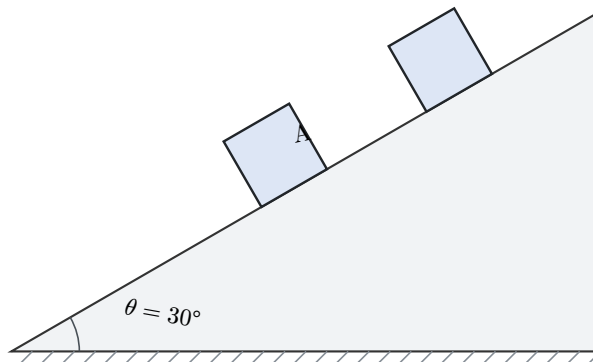
For body labels, "symbol" writes only the declared or derived mass symbol, such as m_A . "mass" writes the symbol, value, and unit, such as $m_A = 4 \text{ kg}$.

```

1  #let s = situation(
2      ramp("incline", angle: 30deg, length: 9),
3      block("A", mass: 4, on: "incline", at: 48%),
4      block("B", mass: 2, on: "incline", at: 76%),
5  )
6  #scene(
7      s,
8      labels: (
9          mode: "name",
10         overrides: (
11             (on: "A", offset: (0.35, 0.3),
12                 rotation: -12deg),
13             (on: "B", visible: false),
14         ),
15     ),
16     angles: (
17         mode: "both",
18         overrides: (
19             (on: "incline", offset: (0.28, 0.18),
20                 rotation: 10deg),
21         ),
22     ),
23 )

```

 Typst

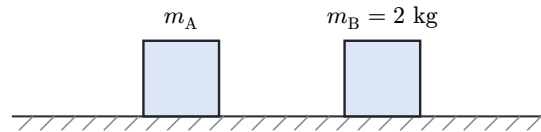


Use a per-body mode when most bodies need their full mass but one needs only its symbol.

```

1  #let s = situation(
2    ground(length: 7),
3    block("A", mass: 4, on: "ground", at:
4      32%),
5    block("B", mass: 2, on: "ground", at:
6      70%),
7  )
8  #scene(
9    s,
10   labels: (
11     mode: "mass",
12     overrides: (
13       (on: "A", mode: "symbol"),
14     ),
15   ),
16 )

```

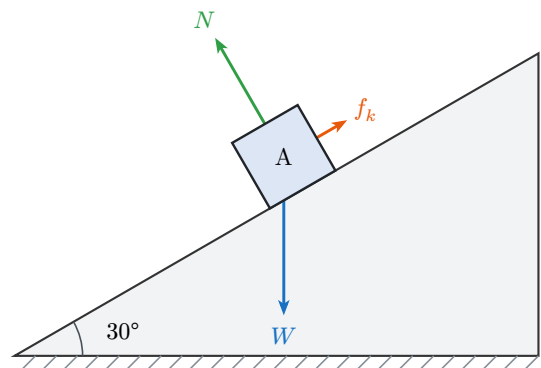


`forces:` draws a body's forces where that body sits, instead of in a diagram of its own. The arrows come from the same enumeration `fbf` uses, so the two agree.

```

1  #let s = situation(
2    ramp("incline", angle: 30deg, length:
3      8),
4    block("A", mass: 2, on: "incline", at:
5      55%,
6    mu: (s: 0.40, k: 0.30)),
7  )
8  #scene(s, forces: "A")

```



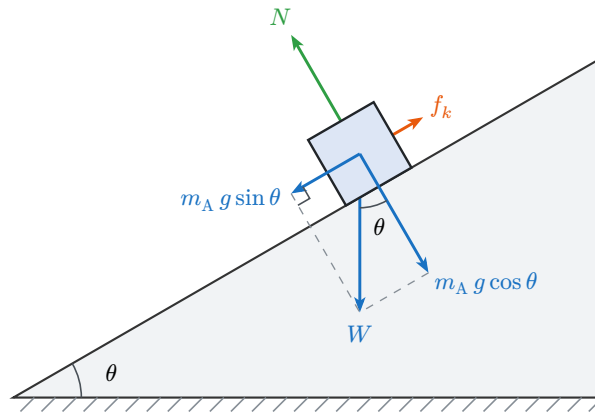
Forces and components in place

`components:` draws the same weight decomposition as `components()`, but keeps the body in the scene. When `forces:` names the same body, the construction closes onto that force view's weight arrow instead of drawing a second one.

```

1 #let s = situation(
2   ramp("incline", angle: 30deg, length: 9),
3   block("A", mass: 4, on: "incline",
4     at: 62%, mu: 0.25),
5 )
6 #scene(s, forces: "A", components: "A",
7   labels: "none", angles: "symbol")

```

 Typst


Note. Force and component arrows need room. Combining them with `labels: "mass"` or `lengths: true` in one small figure can collide; give the figure more `length:` or leave the other labels off.

5.2 fbd()

```

1 #fbd(s, name, axes: auto, outline: true, solve: true, style: (:))

```

 Typst

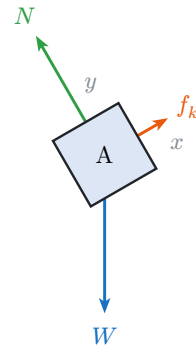
Argument	Meaning
name	Which body to draw. Required.
axes:	Name the surface's axes. <code>auto</code> names them when the surface is inclined.
outline:	Draw the body's outline. With <code>false</code> the arrows meet at a point.
solve:	Solve the body first, so the arrows can be drawn in proportion.
style:	Style overrides for this view.

Arrow lengths are proportional to the forces they stand for. When any magnitude is still symbolic, every arrow falls back to one length.

```

1 #let s = situation(
2   ramp("incline", angle: 30deg, length:
3     6),
4   block("A", mass: 4, on: "incline",
5     at: 55%, mu: (s: 0.40, k: 0.30)),
6   #fbd(s, "A")

```

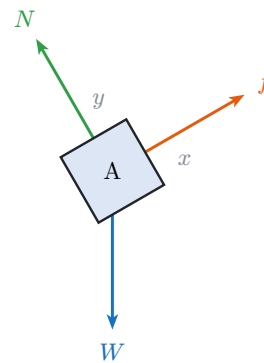


A body the solver cannot handle still gets a diagram, since the forces come from the declaration.

```

1 #let s = situation(
2   ramp("incline", angle: 30deg, length:
3     6),
4   block("A", mass: $m$, on: "incline",
5     mu: (s: $mu_s$, k: $mu_k$)),
6   #fbd(s, "A")

```

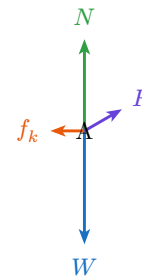


outline: false collapses the body to the point the arrows act on.

```

1 #let s = situation(
2   ground(length: 5, mu: 0.3),
3   block("A", mass: 4, on: "ground"),
4   force(on: "A", magnitude: 15, angle:
5     30deg),
6   #fbd(s, "A", outline: false)

```



5.3 components()

```

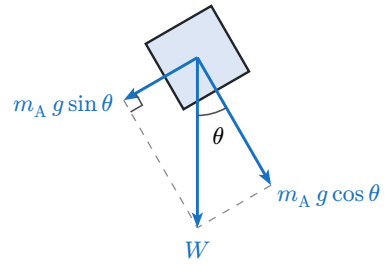
1 #components(s, name, of: "weight", style: (:))

```

Typst

Draws the weight resolved into the surface's own axes, with the construction lines, the right angle, and the inclination arc. of: accepts only "weight" in 0.1.0.

```
1 #let s = situation(  
2   ramp("incline", angle: 30deg, length:  
3     6),  
4   block("A", mass: 4, on: "incline", at:  
5     55%),  
6 )  
7 #components(s, "A")
```

[Typst](#)

6 Answers

Three views read the solution. All of them take an optional body name and an optional `assume`: leave the name out when the situation holds one body.

typed-physics states quantities and stops there. It does not write the argument that connects them, because that argument belongs to your document, in your words and your language.

6.1 `solve()`

```
1 #solve(s, ..name, find: auto, direction: true, assume: auto)
```

 Typst

States one quantity. `find`: chooses which:

Value	What you get
"acceleration"	The acceleration and the direction it points.
"normal"	The normal force.
"friction"	The friction force acting, static or kinetic.
"regime"	Whether the body slides or stays put.
auto	The acceleration for a sliding body, the regime for one that stays put.

`direction: false` drops the words that say which way an acceleration points, leaving the quantity alone for a document that will phrase it itself.

```
1 #let s = situation(
2   ramp("incline", angle: 30deg, length: 6),
3   block("A", mass: 4, on: "incline",
4     at: 55%, mu: (s: 0.40, k: 0.30)),
5 )
6 #solve(s) \
7 #solve(s, find: "normal") \
8 #solve(s, find: "friction") \
9 #solve(s, find: "regime") \
10 #solve(s, find: "acceleration", direction: false)
```

 Typst

$a = 2.36 \text{ m/s}^2$, down the incline
 $N = 34.0 \text{ N}$
 $f_k = 10.2 \text{ N}$
 sliding
 $a = 2.36 \text{ m/s}^2$

Asked for a quantity with no number behind it, `solve` gives the closed form.

```
1 #let s = situation(
2   ramp("incline", angle: 30deg, length: 6),
3   block("A", mass: $m$, on: "incline",
4     mu: (s: $mu_s$, k: $mu_k$)),
5 )
6 #solve(s, assume: "sliding")
```

 Typst

$$a = 4.90 - 8.50 \mu_k \text{ m/s}^2, \text{ down the incline}$$

6.2 force-table()

```
1 #force-table(s, ..name, assume: auto)
```

 Typst

Every force on the body with its components along the surface and out of it, in the order the free-body diagram draws them.

```
1 #let s = situation(
2   ramp("incline", angle: 30deg, length: 6),
3   block("A", mass: 4, on: "incline",
4     at: 55%, mu: (s: 0.40, k: 0.30)),
5 )
6 #force-table(s)
```

 Typst

Force	Magnitude (N)	Along	Out of surface
W	39.2	−19.6	−34.0
N	34.0	0.00	34.0
f_k	10.2	10.2	0.00

6.3 results()

```
1 #results(s, ..name, assume: auto)
```

 Typst

Returns the solution as a dictionary, for pulling a number into your own prose or table.

Key	What it holds
status	"solved" or "undetermined".
body, surface	The names involved.
regime	"static" or "sliding".
assumed	Whether <code>assume:</code> supplied the regime.
rough	Whether the contact has coefficients at all.

normal, friction, acceleration, weight, required, available	Each a dictionary with an expression and its value. friction and acceleration also carry a direction.
motion, downhill	Which way the body would go.

```

1  #let s = situation(
2    ramp("incline", angle: 30deg, length: 6),
3    block("A", mass: 4, on: "incline",
4      at: 55%, mu: (s: 0.40, k: 0.30)),
5  )
6  #let answer = results(s)
7  The block is #answer.regime, the normal force is
8  #calc.round(answer.normal.value, digits: 1) N, and it
9  accelerates at
10 #calc.round(answer.acceleration.value, digits: 2) m/s².

```

 Typst

The block is sliding, the normal force is 34 N, and it accelerates at 2.36 m/s².

6.4 Numbers and symbols

Masses, coefficients, gravity, and force magnitudes take numbers or content. Every answer is carried as far towards a value as the numbers you declared reach: all of them numeric gives a number, some of them gives an expression with those numbers folded in, and none of them gives the closed form.

```

1  #let s = situation(
2    ramp("incline", angle: 30deg, length: 6),
3    block("A", mass: 4, on: "incline",
4      mu: (s: 0.40, k: 0.30)),
5  )
6  #solve(s)

```

 Typst

$a = 2.36 \text{ m/s}^2$, down the incline

A symbolic coefficient leaves only that coefficient standing. The normal force does not depend on it, so it still reaches a number.

```
1 #let s = situation(
2   ramp("incline", angle: 30deg, length: 6),
3   block("A", mass: 4, on: "incline",
4     mu: (s: $mu_s$, k: $mu_k$)),
5 )
6 #solve(s, find: "normal") \
7 #solve(s, assume: "sliding")
```

 Typst

$$N = 34.0 \text{ N}$$

$$a = 4.90 - 8.50 \mu_k \text{ m/s}^2, \text{ down the incline}$$

With nothing numeric left, the answer is an expression in the symbols you declared.

```
1 #let s = situation(
2   gravity: $g$,
3   ramp("incline", angle: 30deg, length: 6),
4   block("A", mass: $m$, on: "incline",
5     mu: (s: $mu_s$, k: $mu_k$)),
6 )
7 #solve(s, assume: "sliding")
```

 Typst

$$a = 0.500 g - 0.866 \mu_k g \text{ m/s}^2, \text{ down the incline}$$

6.5 Choosing the regime with assume:

Whether a body slides is a comparison of two numbers. When either side is symbolic, the four views above say which quantity is missing:

```
1 #let s = situation(
2   ramp("incline", angle: 30deg, length: 6),
3   block("A", mass: $m$, on: "incline",
4     mu: (s: $mu_s$, k: $mu_k$)),
5 )
6 #solve(s)
```

 Typst

typed-physics cannot decide whether **A** slides, because the net force along the surface is symbolic. Give the coefficients and masses as numbers, or state the regime with **assume**:

Pass **assume: "static"** or **assume: "sliding"** to state the regime yourself. **results(s).assumed** records that you supplied it rather than the package deciding, so a document can say as much.

```
1 #let s = situation(  
2   ramp("incline", angle: 30deg, length: 6),  
3   block("A", mass: 4, on: "incline", mu: 0.5),  
4 )  
5 #solve(s, assume: "sliding")
```

 Typst

$a = 0.657 \text{ m/s}^2$, down the incline

6.6 Bodies by name

Name the body when a situation holds more than one. Leaving the name out is an error, not a guess.

```
1 #solve(s, "m2")  
2 #solve(s, "m2", find: "normal", assume: "sliding")  
3 #force-table(s, "m2")  
4 #results(s, "m2")
```

 Typst

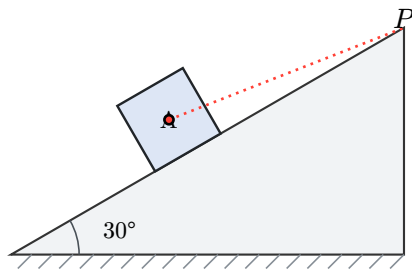
7 Drawing on top

`draw(s)` returns the scene as CeTZ elements instead of a finished canvas, so you can annotate a figure without editing the situation. It takes the same arguments as `scene`. Every body and surface is a named group, and its anchors are addressable coordinates.

The anchors are the ones listed in [Section 4.3](#), plus `start` and `end` on a rope or spring.

```
1  #let s = situation(
2    ramp("incline", angle: 30deg, length: 6),
3    block("A", mass: 4, on: "incline", at: 45%),
4  )
5  #cetz.canvas({
6    import cetz.draw: circle, line, content
7    draw(s)
8    line("A.center", "incline.apex",
9      stroke: (dash: "dotted", paint: red))
10   circle("A.center", radius: 0.06, fill: red)
11   content("incline.apex", $P$, anchor: "south")
12 })
```

t Typst



8 Styling

There are two levels. A **diagram style** sets how a whole figure looks and is accepted by `situation()` and by every view. An **element style** sets how one declared thing looks and is accepted by `block`, the four surfaces, and `force`.

An unknown key is an error at either level, rather than a silent no-op.

8.1 Diagram styles

`situation()` takes the style every view of that situation starts from; a view's own `style:` is merged over it and applies to that view alone.

```
1 #let s = situation(..., style: (body-fill: luma(230), scale: 0.9))
2 #scene(s, style: (force-colors: (applied: red)))
```

 Typst

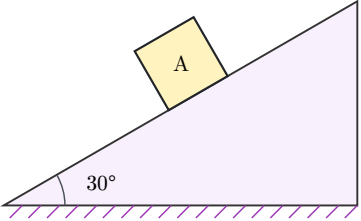
Key	Controls
<code>block-size</code>	Default body edge, in canvas units.
<code>surface-stroke</code> , <code>surface-fill</code>	How a surface is drawn.
<code>hatch-stroke</code> , <code>hatch-spacing</code> , <code>hatch-length</code>	The hatching under a surface.
<code>body-stroke</code> , <code>body-fill</code>	How a body is drawn.
<code>point-mass-fill</code>	The dot a point mass is drawn as.
<code>rope-stroke</code> , <code>spring-stroke</code>	How connectors are drawn.
<code>pulley-fill</code> , <code>pulley-stroke</code>	How a pulley wheel is drawn.
<code>velocity-color</code> , <code>velocity-stroke</code> , <code>velocity-length</code>	How a velocity arrow is drawn.
<code>angle-stroke</code> , <code>angle-radius</code>	The inclination arc.
<code>construction-stroke</code>	Dashed construction lines.
<code>right-angle-size</code>	The right-angle marker in the component view.
<code>length-label-offset</code>	How far off a surface <code>lengths: true</code> writes its length.
<code>dimension-color</code> , <code>dimension-stroke</code> , <code>dimension-extension-stroke</code> , <code>dimension-text</code>	Defaults inherited by scene-only dimensions and their dashed projections.
<code>force-stroke</code>	Thickness of a force arrow.
<code>force-length</code>	Length drawn for the largest force on a body.
<code>force-floor</code>	Shortest an arrow may be drawn, so a small force stays visible.
<code>force-colors</code>	A dictionary keyed <code>weight</code> , <code>normal</code> , <code>friction</code> , <code>applied</code> , <code>tension</code> , <code>component</code> . Merged key by key.
<code>label-text</code> , <code>force-text</code> , <code>angle-text</code>	Text arguments for each kind of label.

scale	Scale factor applied to a finished view.
-------	--

```

1  #let s = situation(
2      ramp("incline", angle: 30deg, length:
3          6),
4      block("A", mass: 4, on: "incline", at:
5          55%),
6      style: (
7          body-fill: rgb("#FFF3BF"),
8          surface-fill: rgb("#F8F0FC"),
9          hatch-stroke: 0.6pt +
10             rgb("#9C36B5"),
11             scale: 0.9,
12         ),
13     )
14 #scene(s)

```

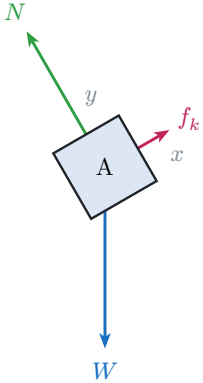


Force colours merge key by key, so naming one leaves the rest alone.

```

1  #let s = situation(
2      ramp("incline", angle: 30deg, length:
3          6),
4      block("A", mass: 4, on: "incline",
5          mu: 0.3),
6      #fbd(s, "A", style: (
7          force-colors: (friction:
8              rgb("#C2255C")),
9          force-length: 1.8,
10         ))

```



The defaults are exported as `theme`, a plain dictionary you can read when building a style of your own.

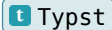
8.2 Element styles

An element's own style wins over the diagram style, for that element and nothing else. Three builders give you argument completion while writing one; each returns a plain dictionary, so you can write the dictionary directly instead.

```

1  #block-style(fill: auto, stroke: auto, label-text: auto)
2  #surface-style(fill: auto, stroke: auto, hatch-stroke: auto,
3                hatch-spacing: auto, hatch-length: auto)
4  #force-style(color: auto, stroke: auto, length: auto, text: auto)

```



Builder	Accepted by
block-style	block, ball, point-mass, disk, ring, rod, pendulum
surface-style	ground, wall, ceiling, ramp, arc
force-style	force, velocity, angular-velocity, torque

connector-style	rope, spring, pulley, pivot, support
block-style	pulley

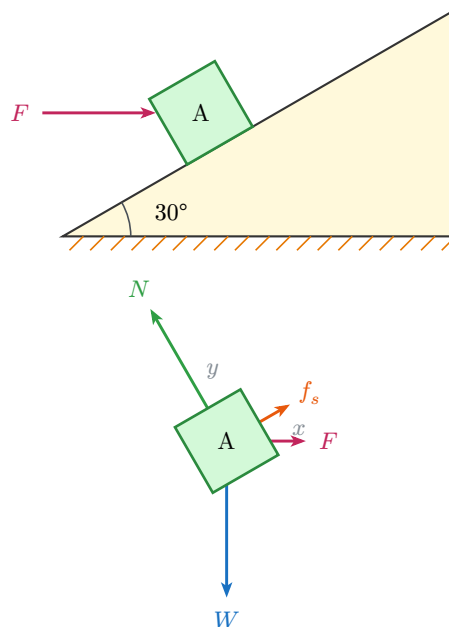
A force's style follows it into the free-body diagram, so an arrow you coloured in the scene is the same colour in every view. `length`: sets how long that arrow is drawn at full size; in a free-body diagram the arrows are still scaled against each other from there.

```

1  #let s = situation(
2    ramp("incline", angle: 30deg, length: 6,
3      style: surface-style(
4        fill: rgb("#FFF9DB"),
5        hatch-stroke: 0.6pt + rgb("#E67700"),
6      )),
7    block("A", mass: 4, on: "incline", at: 40%,
8      mu: 0.3,
9      style: block-style(
10        fill: rgb("#D3F9D8"),
11        stroke: 1pt + rgb("#2B8A3E"),
12      )),
13    force(on: "A", magnitude: 12, angle: 0deg,
14      style: force-style(color: rgb("#C2255C"))),
15  )
16  #scene(s)
17  #fbd(s, "A")

```

 Typst



9 Errors

These stop compilation with a message naming the element and the reason.

If you write	You get
A block reaching past the end of its surface	The overhang, and the two arguments that fix it.
A block with neither on: nor touching:	A request for one of them.
touching: a body declared later	A note that the other body must come first.
solve on a body with no mass:	The name of the missing quantity.
solve on a body resting on a wall or ceiling	What 0.1.0 supports.
A situation whose normal force comes out negative	The value, and that the body leaves the surface.
solve on a situation with several bodies and no name	The list of bodies to choose from.
A zero or negative electrical value	The component and the invalid resistance: , capacitance: , or voltage: value.
A numeric or empty electrical unit:	A request for auto , none , a non-empty string, or content.
A repeated electrical component name	A request to give every source, resistor, and capacitor a unique name.
A route: on some branches of a parallel but not all	A request to route every branch, because the routes together describe the frame.
Two branches taking the same route	The route that was asked for twice.
A cornering branch with more than two components	The number it holds and the two the corner allows.
route: outside a parallel branch	The declaration form that gives the annotation meaning.
fold: true on a load that cannot be split	Which loads have nothing to place on the return rail.
A non-boolean fold:	A request for true , false , or auto .
A non-positive minimum-loop-width	The key and the value that was rejected.
A symbol from the wrong component family	The accepted resistor or voltage-source symbol names.
An unknown style: key	The key that was not recognised.

A regime that cannot be decided is not an error. **solve** reports which quantity is symbolic and how to state the regime instead.

10 What 0.1.0 does not do

The drawing vocabulary is where this package grows. The solver is deliberately still: it balances one body on a ground or a ramp and nothing else.

- Solving anything a connector touches. Ropes, springs, and pulleys are drawn, and a rope tells a free-body diagram that a tension acts, but no tension, acceleration, or contact force between bodies is computed.
- Solving a body supported by a wall or a ceiling, or hanging from anything.
- Action–reaction pairs between touching bodies.
- Force or moment balance for rods, supports, pivots, or applied torques. Those elements draw but do not enter the solver.
- Circular motion on an `arc`, rolling dynamics for `disk` or `ring`, and pendulum motion. Their geometry draws and composes without a derived acceleration.
- `components(of:)` for anything but the weight.
- Solving electrical circuits. The `electricity` namespace draws nested two-terminal series/parallel resistor and capacitor networks but does not calculate resistance, capacitance, charge, current, voltage, impedance, transients, or power.
- Electrical topologies that `series` and `parallel` cannot compose, such as a bridge network. This release also omits electrical components other than ideal voltage sources, resistors, and capacitors.

11 License

typed-physics is released under the MIT license. It is built on [CeTZ](#).